

Week 4 Modularity Lab

Software Construction and Development

Part A Code Archaeology

The original program combines several jobs in `main()`. The comments below label each job before the code is reorganised.

```
def main():
    # INPUT
    name = input("Enter student name: ")
    marks = []
    for i in range(3):
        mark = float(input(f"Enter marks for subject {i + 1}: "))
        # VALIDATION and feedback DISPLAY
        while mark < 0 or mark > 100:
            print("Invalid marks. Enter 0-100.")
            # INPUT again after an invalid mark
            mark = float(input(f"Enter marks for subject {i + 1}: "))
        marks.append(mark)
    # CALCULATION
    total = sum(marks)
    average = total / len(marks)
    # GRADING / BUSINESS RULE
    if average >= 80:
        grade = "A"
    elif average >= 70:
        grade = "B"
    elif average >= 60:
        grade = "C"
    elif average >= 50:
        grade = "D"
    else:
        grade = "F"
    # DISPLAY
    print("\nStudent Result")
    print("-----")
    print("Name:", name)
    print("Total:", total)
    print("Average:", average)
    print("Grade:", grade)
main()
```

Part B Responsibility Map

Concern	Responsibility	Location
Input	Collect the name and three marks, including replacement entries.	main.py
Validation	Check the permitted numeric range.	validation.py
Calculation	Work out the total and mean of the marks.	calculations.py
Grading rule	Assign a letter grade using the average.	calculations.py
Display	Present the calculated student result.	display.py

The range check belongs in `validation.py`. The console retry message remains in `main.py`, which manages user interaction.

Part C Module Design

The solution has four application modules. An additional file, `verify.py`, is used only to test the application.

`main.py` connects the modules. It reads the input, checks each mark, obtains the result values and calls the output function.

`validation.py` decides whether a mark is acceptable and returns a Boolean. This hides the comparison from the calling code.

The total, average and grade functions belong in `calculations.py` because they produce the result values. The average function calls the total function. Neither input prompts nor printing belong here.

`display.py` handles the result's presentation. It accepts the grade rather than working it out, so changing the display cannot change the grading rule.

Original Behaviour

Three marks are collected for each student. The original thresholds are retained: A at 80, B at 70, C at 60 and D at 50; otherwise F. The average is displayed without extra rounding.

The mark check preserves the original comparisons. Text that cannot be converted to a number still causes `ValueError`, as it did before refactoring.

Part D Implementation

Save these files together in the `student_marks` folder. Python 3 and its standard library are sufficient.

`validation.py`

```
def validate_mark(mark):
    """Return True when a mark is within the allowed range."""
    return not (mark < 0 or mark > 100)
```

`calculations.py`

```
def calculate_total(marks):
    return sum(marks)

def calculate_average(marks):
    return calculate_total(marks) / len(marks)

def calculate_grade(average):
    if average >= 80:
        return "A"
    elif average >= 70:
        return "B"
    elif average >= 60:
        return "C"
    elif average >= 50:
        return "D"
    return "F"
```

`display.py`

```
def display_result(name, total, average, grade):
    print("\nStudent Result")
    print("-----")
    print("Name:", name)
    print("Total:", total)
    print("Average:", average)
    print("Grade:", grade)
```

main.py

```
from validation import validate_mark
from calculations import (
    calculate_total, calculate_average, calculate_grade
)
from display import display_result
```

```
def main():
    name = input("Enter student name: ")
    marks = []
    for i in range(3):
        prompt = f"Enter marks for subject {i + 1}: "
        mark = float(input(prompt))
        while not validate_mark(mark):
            print("Invalid marks. Enter 0-100.")
            mark = float(input(prompt))
        marks.append(mark)

    total = calculate_total(marks)
    average = calculate_average(marks)
    grade = calculate_grade(average)
    display_result(name, total, average, grade)
```

```
if __name__ == "__main__":
    main()
```

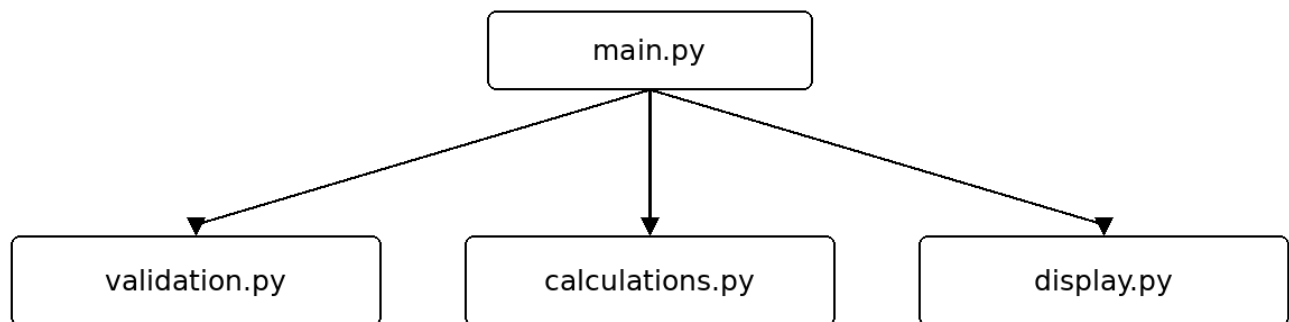
Running the Project

```
cd student_marks
python main.py
```

```
# Run the supplied checks
python verify.py
```

Using the main guard allows tests to import main.py without starting the prompts. A rejected entry is retried for the same subject.

Part E Dependency Mapping



The arrows represent module dependencies. Only main.py imports the three helper modules. None of those modules imports main.py or another helper module.

The display function only needs the completed values. The calculation functions only need their inputs, not the details of the output layout.

Moving to a GUI would change the presentation and the input coordination. The existing validation and calculation functions could still be called by the new interface.

main.py needs its imports to run the workflow. A dependency between calculations.py and display.py would add no useful service and would join concerns that should stay separate.

Part F Change Request Test

Change	Module affected	Reason
A ≥ 85, B ≥ 75, C ≥ 65, D ≥ 55	calculations.py	Only calculate_grade contains the thresholds.
Result format changes	display.py	display_result owns labels and layout.
Validation message changes	main.py	The retry loop prints the feedback message.
GUI output replaces console output	display.py; main.py for GUI wiring	Replace presentation and adapt interaction; reuse calculations and validation.

The revised thresholds are a proposed change, not part of the baseline code. With those thresholds, 82 would receive B rather than A. The sum and average would be unaffected.

Only the wording of the retry message changes in main.py. A change to the valid range would instead be made in validation.py.

Part G Verification

The checks below cover calculation results, grade cutoffs, valid range limits and retry behaviour.

Test	Input	Verified result
Normal student	75, 82, 68	Total 225; average 75; grade B
Grade boundaries	50, 60, 70, 80	D, C, B, A respectively
Just below boundaries	49.99, 59.99, 69.99, 79.99	F, D, C, B respectively
Valid range limits	0 and 100	Both accepted
Invalid attempts	-5 and 105	Rejected; same subject is prompted again
Second student	40, 45, 35	Total 120; average 40; grade F

Actual Console Output

This example includes two invalid entries. Each is replaced before the result is calculated.

```
Enter student name: Ali
Enter marks for subject 1: -5
Invalid marks. Enter 0-100.
Enter marks for subject 1: 75
Enter marks for subject 2: 105
Invalid marks. Enter 0-100.
Enter marks for subject 2: 82
Enter marks for subject 3: 68
```

Student Result

```
-----
Name: Ali
Total: 225.0
Average: 75.0
Grade: B
```

Second student inputs: Sara; 40; 45; 35.

Enter student name: Sara

Enter marks for subject 1: 40

Enter marks for subject 2: 45

Enter marks for subject 3: 35

Student Result

Name: Sara

Total: 120.0

Average: 40.0

Grade: F

The five automated checks passed. In a separate comparison using 13 cases, both versions printed the same results.

Reflection on the Design

A function gives one task a name and a reusable body. A module groups related functions in a file, creating a boundary around a responsibility.

The most focused module is `validation.py`: its code is concerned only with deciding whether a mark is accepted.

Calling `calculate_grade` makes `main.py` depend on that function's interface. It needs the correct name and argument, but does not need to know the threshold comparisons.

Separate modules reduce the amount of code affected by a change. A new grade policy belongs in `calculations.py`; a new output layout belongs in `display.py`.

This is a reasonable level of separation for a small program. The validation module is short but has a useful independent purpose. Adding input or storage to `calculations.py` would weaken its cohesion.